MANUAL DE USUARIO

TaskScript - Analizador Léxico y Sintáctico para Tableros Kanban

Universidad San Carlos de Guatemala
Facultad de Ingeniería, Escuela de Ciencias y Sistemas
Curso: Lenguajes Formales y Programación
Proyecto 2 - Primer Semestre 2026

Tabla de Contenidos

- Introducción
- Requisitos del Sistema
- Instalación y Compilación
- Estructura del Lenguaje TaskScript
- Ejemplos Completos
- Uso de la Aplicación
- Interpretación de la Tabla de Tokens
- Interpretación de Mensajes de Error
- Reportes HTML Generados
- Árbol de Derivación con Graphviz
- Solución de Problemas Comunes
- Datos de Prueba

Introducción

¿Qué es TaskScript?

TaskScript es una herramienta de análisis léxico y sintáctico diseñada para interpretar archivos de texto con estructura de tableros Kanban. La aplicación permite:

- Analizar archivos con extensión .task
- Validar la estructura léxica y sintáctica del lenguaje
- Generar reportes HTML con visualización del tablero
- Visualizar el árbol de derivación mediante Graphviz

¿Para qué sirve?

TaskScript es útil para equipos de desarrollo que desean definir tableros Kanban en archivos de texto plano, tambien validar automáticamente la estructura de sus proyectos donde se pueden generar reportes visuales para reuniones de seguimiento tambine en la identificacion de errores en la definición de tareas y columnas

Componentes del Sistema

Componente	Tecnología	Función
Backend	C++17	Análisis léxico y sintáctico
Frontend	React	Interfaz gráfica de usuario
Servidor HTTP	Winsock	Comunicación backend-frontend
Reportes	HTML/CSS	Visualización de resultados

Requisitos del Sistema

Hardware Requerido

Componente	Mínimo	Recomendado
Procesador	Intel Core i3	Intel Core i5 o superior
RAM	4 GB	8 GB
Almacenamiento	500 MB	1 GB

Software Requerido

Software	Versión	Propósito
Windows	10/11 (64 bits)	Sistema operativo
MinGW	8.0+	Compilador C++
Node.js	18.x o superior	Ejecutar React
NPM	9.x o superior	Manejador de paquetes
Git	2.x	Control de versiones
Navegador	Chrome/Edge/Firefox	Visualizar aplicación

Instalación de Dependencias Previas

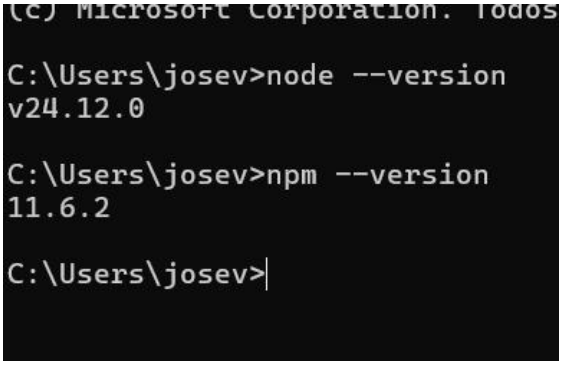
Instalar MinGW (si no está instalado):

Descargar MinGW desde https://osdn.net/projects/mingw/
Ejecutar el instalador y seleccionar mingw32-gcc-g++
Agregar C:\MinGW\bin al PATH del sistema

Reiniciar la terminal
Instalar Node.js (si no está instalado):
Descargar Node.js desde https://nodejs.org/

Ejecutar el instalador (versión LTS recomendada)

Verificar con node --version y npm --version



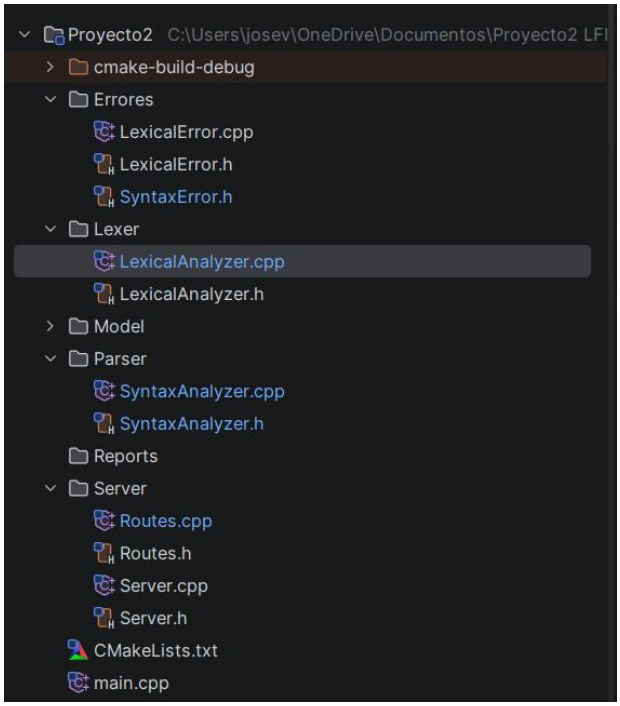
Terminal mostrando versiones instaladas
Muestra: g++ --version, node --version, npm --version

Instalación y Compilación

Obtener el Código Fuente
bash
git clone https://github.com/joseD2023/LFP_P2_1S2026_202407095

Terminal mostrando clonación exitosa para obtener el repositorio completamente funcionando.

Estructura de Carpetas del Proyecto



Instalación y Compilación

Compilar el Backend (C++)

Primeramente necesitamos un IDE para c++ para este caso tenemos CLION y su descarga en su pagina principal.

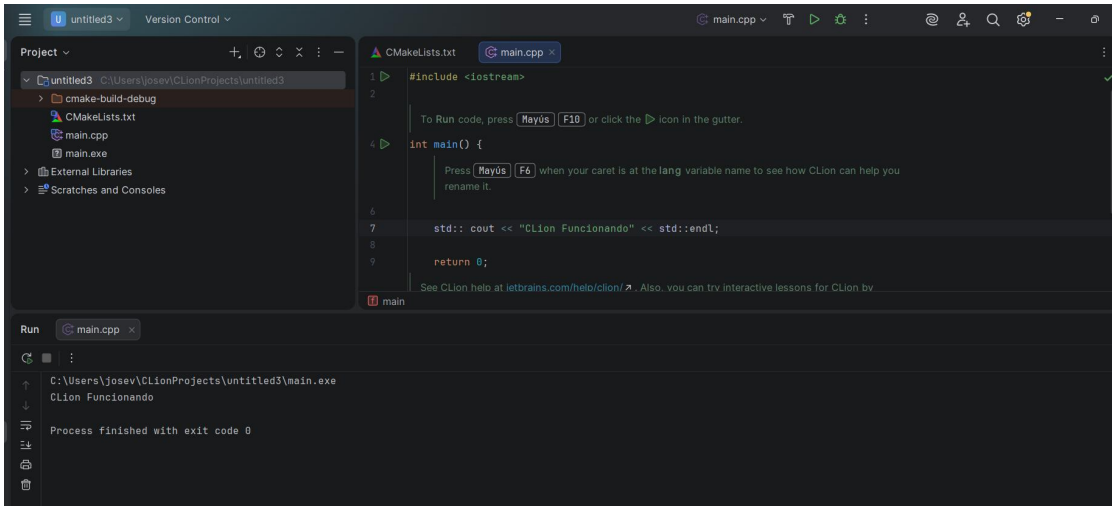
https://www.jetbrains.com/clion/download/?section=windows

Cuando ya instalamos CLion entonces comenzamos.

En CLion:
Abrir CLion

Ir a File > Open y seleccionar la carpeta backend
Esperar a que CLion cargue el proyecto

Presionar Ctrl + F9 (Build)
Presionar Shift + F10 (Run)



Funcionamiento del Frontend (React)

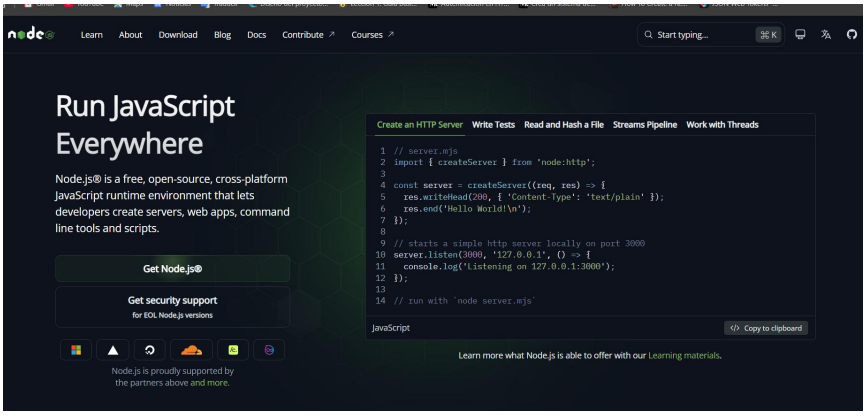
Instalar Node.js (Requisito Previo)

Node.js es necesario para ejecutar el frontend de React.

¿Cómo instalar Node.js? descargar Node.js

Ir a: https://nodejs.org/

Descargar la versión LTS (recomendada para la mayoría)



El archivo se llama algo como: node-v20.11.0-x64.msi

Página de descarga de Node.js con el botón "LTS" marcado
Instalar Node.js

Ejecutar el archivo .msi descargado

Hacer clic en "Next" hasta terminar

Aceptar todo por defecto (no cambiar nada)

Al final, hacer clic en "Install" y luego "Finish"

Instalador de Node.js - Aceptando términos, despues debemos verificar que se instaló bien

```
C:\Users\josev>node --version
v24.12.0
```

La varificacion que se instalo correctamente.

Abrir el proyecto en VS Code o en la terminal

bash
cd Proyecto2/frontendnpm installnpm start o tambien podemos utilizar la parte de Acceder al directorio de donde descargamos el proyecto y usamos el comando: npm run dev para hacer el funcionamiento de nuestra web.

Nota: npm install solo se ejecuta LA PRIMERA VEZ que abres el proyecto.
Terminal con el proyecto corriendo
Abrir el navegador

Automáticamente se abre http://localhost:5173

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL
TERMINAL

  VITE v8.0.10  ready in 594 ms

  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

Estructura del Archivo .task

```
TABLERO "Nombre del Tablero" {
    COLUMNA "Nombre Columna" {
        tarea: "Nombre Tarea" [prioridad: ALTA, responsable: "Persona", fecha_limite: AAAA-MM-DD]
    };
};
```

Reglas Rápidas

Regla	Correcto	Incorrecto
tarea necesita :	tarea: "Hola"	tarea "Hola"
Texto entre comillas	"Mi texto"	Mi texto
Las tareas se separan con ,	tarea: "A", tarea: "B"	tarea: "A" tarea: "B"
Cada columna termina con };	};	}
Prioridades en Mayusculas	ALTA	alta

Prioridades

Prioridad	Color
ALTA	Rojo
MEDIA	Amarillo
BAJA	Verde

Ejemplos de Archivos

Tablero valido

```
TABLERO "Proyecto Ventas" {
    COLUMNA "Por Hacer" {
        tarea: "Analizar requerimientos" [prioridad: ALTA, responsable: "Ana", fecha_limite: 2026-06-10],
        tarea: "Diseñar base de datos" [prioridad: ALTA, responsable: "Luis", fecha_limite: 2026-06-15]
    };
    COLUMNA "Completado" {
        tarea: "Configurar repositorio" [prioridad: BAJA, responsable: "Carlos", fecha_limite: 2026-05-30]
    };
};
```

Tablero Invalido

```
TABLERO "Proyecto&Errores" {
    COLUMNA "Backlog" {
        tarea "Sin dos puntos" [prioridad: ALTA, responsable: "Ana_Luis", fecha_limite: 2026-06-01]
    };
};
```

Errores que detecta:

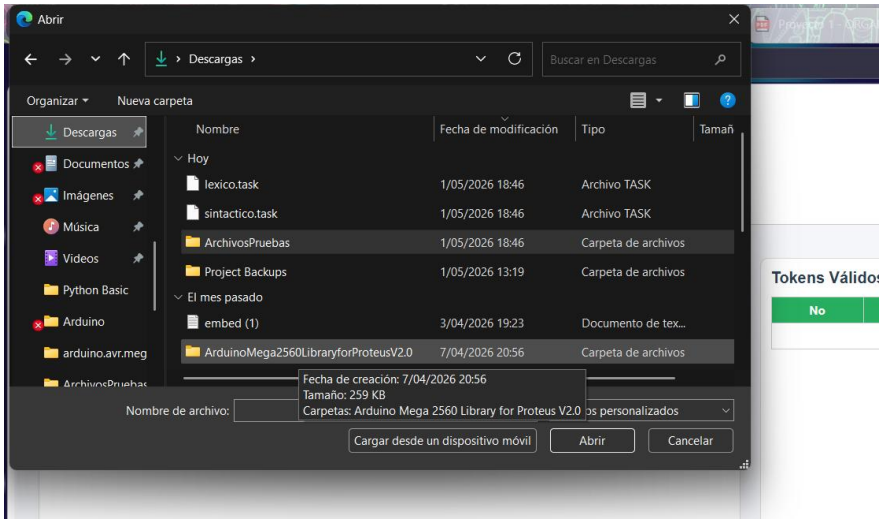
Línea 3: Falta : después de tarea
Línea 3: Caracter & en el nombre
Línea 4: Caracter _ en "Ana_Luis"

Cómo Usar la Aplicación

Cargar un Archivo

Debemos ir a Nuestra pagina principal, tenemos dos opciones:
Clic en "Archivo" Seleccionar archivo .task o tambien copiar y pegar el código en el textarea.

En esta ocasion lo haremos atraves de cargando archivo desde el botón



Seleccionamos el archivo analizar y luego Hacemos Clic en "Analizar"

Se muestraran tabla de Errores (izquierda) t abla de Tokens (derecha)

Le damos analizar al boton de abajo.

Errores del Análisis

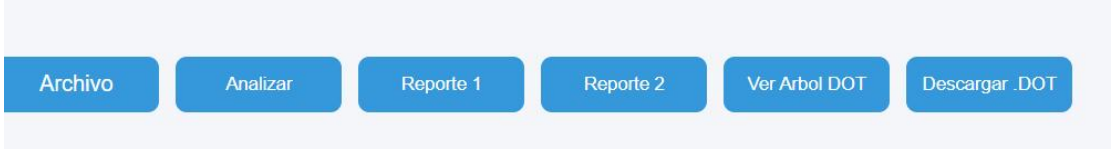
No	Lexema	Tipo	Descripcion	Fila	Columna
1	@	Léxico	El caracter '@' no se puede usar dentro del texto	5	76
2	$	Léxico	El caracter '$' no se puede usar dentro del texto	6	86
3	#	Léxico	El caracter '#' no se puede usar dentro del texto	11	43
4	&	Léxico	El caracter '&' no se puede usar dentro del texto	11	90
5	~	Léxico	El caracter '~' no se puede usar dentro del texto	17	78
6	#	Léxico	El caracter '#' no se puede usar dentro del texto	18	30
7	^	Léxico	El caracter '^' no se puede usar dentro del texto	24	81

Tokens Válidos

No	Lexema	Tipo	Fila	Columna
1	TABLERO	TABLERO	1	1
2	"Sistema de Ventas Q2-2026"	STRING	1	9
3	{	LLAVE_ABRE	1	37
4	COLUMNA	COLUMNA	2	5
5	"Backlog"	STRING	2	13
6	{	LLAVE_ABRE	2	23
7	tarea	TAREA	3	9
8	:	DOS_PUNTOS	3	14
9	"Analizar requerimientos del cliente"	STRING	3	16
10	[	CORCHETE_ABRE	3	54
11	prioridad	PRIORIDAD	3	55
12	:	DOS_PUNTOS	3	64
13	ALTA	ALTA	3	66

Generar Reportes

Botón	Qué hace
Reporte 1	Muestra el tablero Kanban (columnas y tarjetas)
Reporte 2	Muestra carga por responsable (con barras)
Ver Árbol DOT	Muestra código DOT del árbol
Descargar .DOT	Descarga archivo .dot



Reporte 1.

Reporte 2.

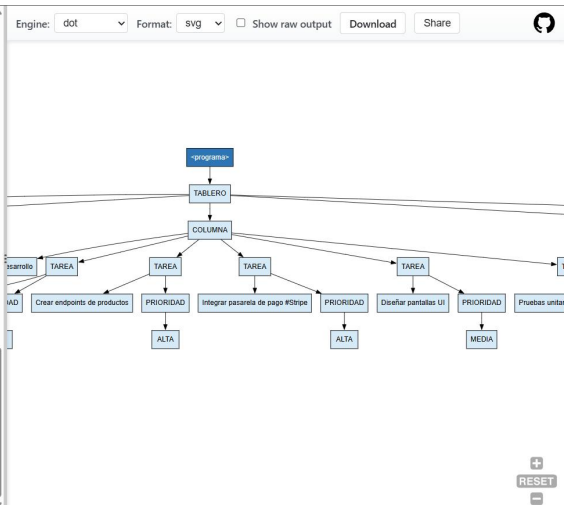
Reporte 2 - Carga por Responsable

Responsable	Total	ALTA	MEDIA	BAJA	%
Andrea	3	3	0	0	
Carlos	1	0	0	1	
Carlos&Backend	1	1	0	0	
Carlos^Ops	1	0	1	0	
Lucia	1	1	0	0	
Lucia@Dev	1	0	1	0	
Marco	2	0	1	1	
Marco$Admin	1	0	0	1	
Roberto	3	2	1	0	
Team~QA	1	1	0	0	
Valeria	1	0	1	0	

Arbol DOT: Copialo y Pegalo en Graphviz

```
digraph ArbolDerivacion {
rankdir=TB;
node [shape=box, style=filled, fontname="Arial"];
n0 [label="<programa>", fillcolor="#2E75B6", fontcolor=white];
n1 [label="TABLERO", fillcolor="#D6EAF8"];
n0 -> n1;
n2 [label="Sistema de Ventas Q2-2026", fillcolor="#D6EAF8"];
n1 -> n2;
n3 [label="COLUMNA", fillcolor="#D6EAF8"];
n1 -> n3;
n4 [label="Backlog", fillcolor="#D6EAF8"];
n3 -> n4;
n5 [label="TAREA", fillcolor="#D6EAF8"];
n3 -> n5;
n6 [label="Analizar requerimientos del cliente", fillcolor="#D6EAF8"];
n5 -> n6;
n7 [label="PRIORIDAD", fillcolor="#D6EAF8"];
n5 -> n7;
n8 [label="ALTA", fillcolor="#D6EAF8"];
n7 -> n8;
```

```
n52 [label="ALTA", fillcolor="#D6EAF8"];
n51 -> n52;
n53 [label="TAREA", fillcolor="#D6EAF8"];
n43 -> n53;
n54 [label="Corregir bug #4471 en carrito", fillcolor="#D6EAF8"];
n53 -> n54;
n55 [label="PRIORIDAD", fillcolor="#D6EAF8"];
n53 -> n55;
n56 [label="MEDIA", fillcolor="#D6EAF8"];
n55 -> n56;
n57 [label="COLUMNA", fillcolor="#D6EAF8"];
n1 -> n57;
n58 [label="Completado", fillcolor="#D6EAF8"];
n57 -> n58;
n59 [label="TAREA", fillcolor="#D6EAF8"];
n57 -> n59;
n60 [label="Setup inicial del proyecto", fillcolor="#D6EAF8"];
n59 -> n60;
n61 [label="PRIORIDAD", fillcolor="#D6EAF8"];
n59 -> n61;
n62 [label="BAJA", fillcolor="#D6EAF8"];
n61 -> n62;
n63 [label="TAREA", fillcolor="#D6EAF8"];
n57 -> n63;
n64 [label="Configurar GitHub Actions", fillcolor="#D6EAF8"];
n63 -> n64;
n65 [label="PRIORIDAD", fillcolor="#D6EAF8"];
n63 -> n65;
n66 [label="BAJA", fillcolor="#D6EAF8"];
n65 -> n66;
n67 [label="TAREA", fillcolor="#D6EAF8"];
n57 -> n67;
n68 [label="Definir arquitectura del sistema", fillcolor="#D6EAF8"];
n67 -> n68;
n69 [label="PRIORIDAD", fillcolor="#D6EAF8"];
n67 -> n69;
n70 [label="ALTA", fillcolor="#D6EAF8"];
n69 -> n70;
n71 [label="TAREA", fillcolor="#D6EAF8"];
n57 -> n71;
n72 [label="Levantar entorno Docker", fillcolor="#D6EAF8"];
n71 -> n72;
n73 [label="PRIORIDAD", fillcolor="#D6EAF8"];
n71 -> n73;
n74 [label="MEDIA", fillcolor="#D6EAF8"];
n73 -> n74;
}
```



Cómo Interpretar los Resultados

Tabla de Errores

Columna	Qué significa
No	Número del error
Lexema	Texto que causó el error
Tipo	Léxico (caracter malo) o Sintáctico (estructura mala)

Columna	Qué significa
Descripción	Explicación clara del problema
Fila	Línea donde está el error
Columna	Posición exacta

Ejemplo:

No	Lexema	Tipo	Descripción	Fila	Columna
1	"Ana&Luis"	Léxico	El caracter '&' no se puede usar dentro del texto	5	25

Tabla de Tokens

Columna	Qué significa
No	Número del token
Lexema	Texto encontrado
Tipo	Clasificación (TABLERO, STRING, etc.)
Fila	Línea donde aparece
Columna	Posición

Mensajes de Error Comunes

Mensaje	Qué significa	Cómo arreglarlo
El caracter '&' no se puede usar	Usaste & en un texto	Cambia & por y
Se esperaba ':' pero se encontró 'STRING'	Falta : después de tarea	Escribe tarea:
Se esperaba ';' pero se encontró '}'	Falta ; al final de columna	Agrega ; antes de }

Reporte 1 - Tablero Kanban

Muestra columnas con tarjetas:

Tarjeta roja = Prioridad ALTA
Tarjeta amarilla = Prioridad MEDI
Tarjeta verde = Prioridad BAJA

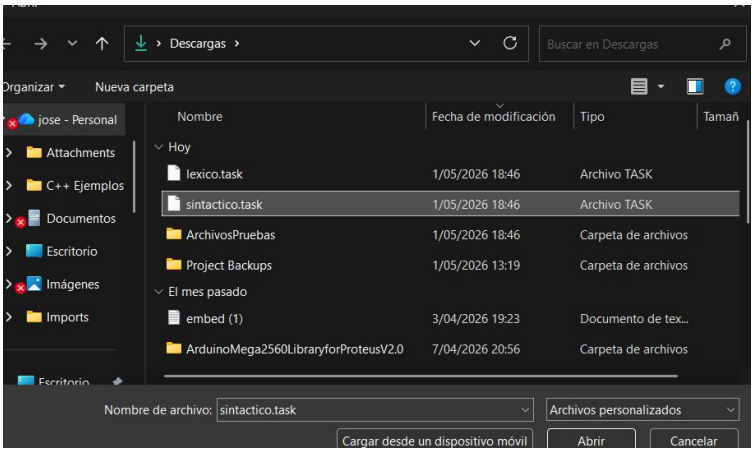
Reporte 1 con colores
Reporte 2 - Carga por Responsable muestra tabla con:

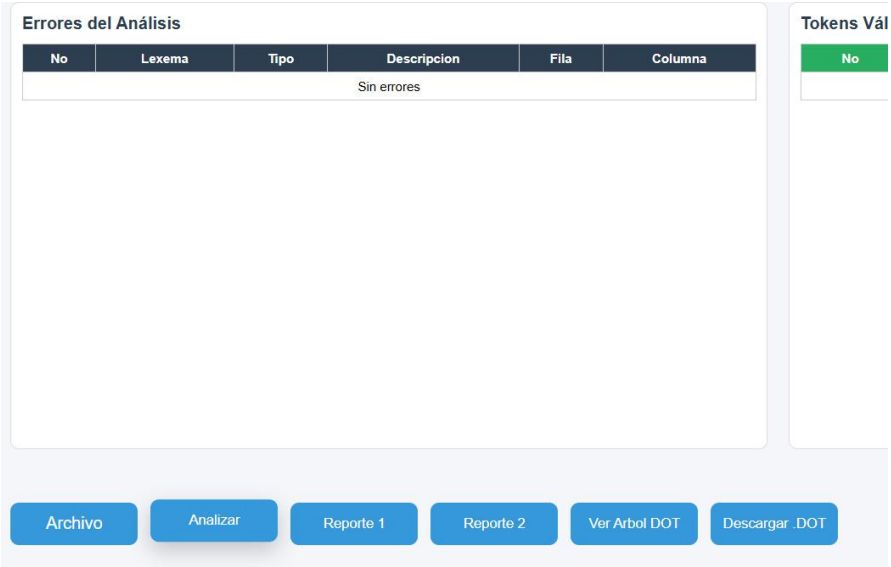
- Nombre del responsable
- Total de tareas
- Cuántas de cada prioridad
- Porcentaje con barra visual

Ejemplo de Error Sintactico

Los paso siguen siendo los mismo solo que ahora vamos con lo que son el analisis sintactico.

Vamos a cargar un archivo con error sintactico.





Le damos analizar.

Errores del Análisis

No	Lexema	Tipo	Descripcion	Fila	Columna
1	;	Sintáctico	Se esperaba '}' pero se encontró 'PUNTO_COMA'	8	5

Tokens Válidos

No	Lexema	Tipo	Fila	Columna
1	TABLERO	TABLERO	1	1
2	"Migración a la Nube - Infraestructura"	STRING	1	9
3	{	LLAVE_ABRE	1	50
4	COLUMNA	COLUMNA	2	5
5	"Planificación"	STRING	2	13
6	{	LLAVE_ABRE	2	30
7	tarea	TAREA	3	9
8	:	DOS_PUNTOS	3	14
9	"Inventario de servicios actuales"	STRING	3	16
10	[	CORCHETE_ABRE	3	51
11	prioridad	PRIORIDAD	3	52
12	:	DOS_PUNTOS	3	61
13	ALTA	ALTA	3	63

Vemos como nos aparece un error sintactico porque el programa encontro una incosistencia en lo que es el error sintactico mas no lexico.

Donde el reporte 1 No se va a generar por la estructura no fue la correcta.

Archivo | Analizar | Reporte 1 | Reporte 2 | Ver Arbol DOT | Descargar .DOT

Reporte 2 - Carga por Responsable

Responsable	Total	ALTA	MEDIA	BAJA	%
Arquitecto Cloud	2	2	0	0	
CTO	1	1	0	0	
DBA	2	2	0	0	
DevOps	3	1	2	0	
FinOps	1	1	0	0	
Network Admin	1	1	0	0	
QA Lead	1	1	0	0	
RRHH	1	0	1	0	
SRE	1	1	0	0	

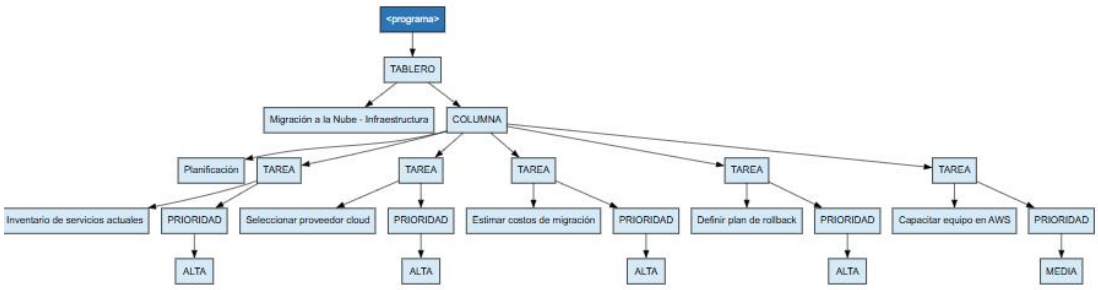
El reporte 2 si se genera ya que no depende directamente del parser para generarse entonces encontraremos la parte del reporte 2.

Ahora nos vamos al ver arbol .DOT para generar el arbol

Arbol DOT: Copialo y Pegalo en Graphviz

```
digraph ArbolDerivacion {
rankdir=TB;
node [shape=box, style=filled, fontname="Arial"];
n0 [label="<programa>", fillcolor="#2E75B6", fontcolor=white];
n1 [label="TABLERO", fillcolor="#D6EAF8"];
n0 -> n1;
n2 [label="Migración a la Nube - Infraestructura", fillcolor="#D6EAF8"];
n1 -> n2;
n3 [label="COLUMNA", fillcolor="#D6EAF8"];
n1 -> n3;
n4 [label="Planificación", fillcolor="#D6EAF8"];
n3 -> n4;
n5 [label="TAREA", fillcolor="#D6EAF8"];
n3 -> n5;
n6 [label="Inventario de servicios actuales", fillcolor="#D6EAF8"];
n5 -> n6;
n7 [label="PRIORIDAD", fillcolor="#D6EAF8"];
n5 -> n7;
n8 [label="ALTA", fillcolor="#D6EAF8"];
n7 -> n8;
```

Tenemos el .Dot ahora nos vamos a Graphiz Online para Graficar nuestro arboles



Y ahora tenemos los 3 reportes para que podamos observar las circustancia de cada parte de nuestro proyecto.

Solución de Problemas

Problema	Solución
El servidor no inicia	Verificar que el puerto 8080 esté libre
La página no carga	Ejecutar npm start en frontend
Errores de compilación	Verificar que tienes MinGW instalado
No se ven los reportes	Analizar primero un archivo válido sin errores
Caracteres rato en la tabla	No usar tildes o caracteres especiales